Python による時間枠割当ソルバの解説

希望順位付き区間スケジューリング

数学的定式化・動的計画法・正当性の証明

Hiroki Funashima

Thursday, October 22, 2026

本資料は、添付された単一ファイルの PyQt6 アプリケーションを題材として、時間区間の重複制約、希望順位の辞書式目的関数、割当済み参加者のビットマスク表現、および区間スケジューリングの動的計画法を、定義・実例・証明によって詳述する。原実装の振る舞いと、改善案・一般化した数理モデルを明確に区別して扱う。

目次

1 問題の全体像 4

1.1 何を解くプログラムか 4

1.2 処理の層と責任範囲 4

1.3 GUI と計算核を分離する意義 4

2 入力データの数学的表現 5

2.1 集合と添字 5

2.2 希望順位行列 5

2.3 配列の整合性 6

3 割当問題の厳密な定式化 6

3.1 二値変数 6

3.2 一人一枠制約 6

3.3 共有資源の重複禁止 6

3.4 割当を評価する四次元ベクトル 7

3.5 辞書式順序（lexicographic order）の意味 7

3.6 重み付き和との違い 8

4 区間スケジューリングと動的計画法 8

4.1 なぜ単純な貪欲法では不足するのか 8

4.2 終了時刻順へのソート 8

4.3 直前の両立可能位置 8

4.4 参加者集合をビットマスクで表す 9

4.5 DP 状態の厳密な意味 10

4.6 区間一件を採用するときの増分 10

4.7 DP の基本漸化式 10

4.8 正方向の更新として見る 11

5 Python 実装と数学記号の対応 11

5.1 前処理と入力検査 11

5.2 DP テーブルの初期化 11

5.3 $p(j)$ に対応する探索 12
5.4 状態遷移と辞書式比較 12
5.5 最終解と復元 12

6 実際に手で追う計算例 13
6.1 入力例と両立位置 13
6.2 第 1 段階：枠 1 まで 13
6.3 第 2 段階：枠 2 まで 14
6.4 第 3 段階：枠 3 まで 14
6.5 第 4 段階：枠 4 まで 14
6.6 最終段階：枠 5 まで 15
6.7 重要な教訓：局所的に最善とは限らない 15

7 厳密最適性の証明 15
7.1 準備：延長可能性を決める情報 15

8 計算量とスケーラビリティ 16
8.1 状態数 16
8.2 時間計算量 16
8.3 人数 15 人という制限の意味 17
8.4 同じ厳密性を保った改善案 17

9 画面処理・ファイル処理の読み方 17
9.1 Qt のイベント駆動 17
9.2 枠の一括生成 18
9.3 希望の可視化と実際の割当は別物 18
9.4 JSON 保存と復元 18

10 入力検証・限界・モデル拡張 18
10.1 現在の実装が明示的に保証すること 18
10.2 現在の実装が保証しないこと 19
10.3 入力検査に関する実装上の注意 19
10.4 公平性・必須対象者・時間的余裕の導入（拡張案） 19

11 検証方法：厳密解の独立確認 20
11.1 小規模な全列挙との照合 20

11.2 最低限確認したい境界事例 20

12 より一般的な視点とまとめ 20

付録 A 記号一覧 21

付録 B 読解する際のソースコード位置 22

1 問題の全体像

1.1 何を解くプログラムか

このプログラムは、単一の共有資源（例えば面談を担当する一人の教員）について、日付付き時間枠を複数の参加者へ割り当てる。参加者は時間枠ごとに第 1 希望、第 2 希望、第 3 希望、希望なしのいずれかを入力する。時間が重複しない範囲で、各参加者に**高々一つ**の時間枠を割り当て、まず割当人数を最大化し、次に希望順位の良い割当を選択する。

同じ日時の枠だけでなく、`09:00--10:00` と `09:30--10:30` のような**異なる行の部分的重複**も許さない。ただし、終了時刻と次の開始時刻が一致する場合は両方を利用できる。

1.2 処理の層と責任範囲

主な構成を表 1 に示す。

表 1: 実装の主要構成

機能	名前	役割
入力の表現	`people, dates, slots, wishes`	参加者、日時、希望順位行列を管理する
結果型	`Solution`	割当組、未割当者、希望順位件数を保持する
日時の解釈	`_bounds()`	日付と時刻文字列から半開区間の境界を作る
厳密最適化	`solve()`	終了時刻順の区間 DP とビットマスクで最適化する
個別・一括編集	`SlotDialog, BatchSlotDialog`	日付・開始終了時刻を入力し、連続枠を生成する
画面とファイル	`Scheduler`	表示、希望編集、実行、JSON 入出力を管理する
起動	`main()`	Qt イベントループを開始する

1.3 GUI と計算核を分離する意義

`solve(people,dates,slots,wishes)` は PyQt6 のウィジェットを参照しない。したがって、入力を通常の Python のリストで与えれば、GUI を起動しなくても最適化を試験できる。画面表示に関する変更と、アルゴリズム自体の変更を独立に行えるという、設計上の明確な利点がある。

2　入力データの数学的表現

2.1　集合と添字

参加者を

$$P = \{0, 1, \dots, n-1\}$$

とし、入力の時間枠を

$$I = \{0, 1, \dots, \ell - 1\}$$

とする。ここで n は参加者数、ℓ は入力枠数である。プログラムでは、希望がすべてゼロの行を最適化前に除外するので、以下では**希望のある時間枠**を改めて終了時刻順に $1, \dots, m$ と番号付けして用いる（$m \le \ell$）。

それぞれの時間枠 i の開始・終了時刻を s_i, e_i とする。

定義 2.1 (半開時間区間)**.** 時間枠 i が占有する時間の集合を

$$J_i = [s_i, e_i) = \{t \mid s_i \le t < e_i\}$$

と定義する。ただし $s_i < e_i$ とする。

終了時刻を含まないため、[09:00, 09:30) と [09:30, 10:00) は重ならない。二つの区間 i, j が両立する必要十分条件は

$$J_i \cap J_j = \varnothing \quad \Longleftrightarrow \quad e_i \le s_j \text{ または } e_j \le s_i$$

である。

2.2　希望順位行列

希望を

$$w_{ip} \in \{0, 1, 2, 3\}$$

とする。$w_{ip} = 0$ は希望なし、$1, 2, 3$ はそれぞれ第 1、第 2、第 3 希望である。原コードでは、`wishes[i][p]` として格納される。つまり、**行が時間枠、列が参加者**である。

例 2.2 (希望行列)**.** 3 人 A、B、C について、次の入力を考える。

時間枠	A	B	C
09:00–09:30	1	0	0
09:15–09:45	0	1	0
09:30–10:00	0	2	1
10:00–10:30	1	1	2
10:30–11:00	0	0	1

例えば第 3 行の $w_{3,B} = 2$ は、B がその時間枠を第 2 希望とすることを表す。

同じ参加者が複数の時間枠に同一の希望順位を入力することは、**原実装では許される**。「第 1 希望は一つだけ」という制約は、このプログラムには存在しない。

2.3 配列の整合性

`dates`, `slots`, `wishes` の同じ行番号は同一時間枠を指す。`sort_slots()` がこの 3 配列をまとめて `zip()` し、日時順に並べ替えるのは、ソート後にも希望の所有関係を保つためである。1 配列だけを並べ替えると、希望者と日時の対応が破壊される。

3 割当問題の厳密な定式化

3.1 二値変数

二値決定変数を

$$x_{ip} = \begin{cases} 1, & \text{時間枠 } i \text{ を参加者 } p \text{ に割り当てる}, \\ 0, & \text{そうでない} \end{cases}$$

と定める。希望のない組合せは採用してはならない。

$$x_{ip} = 0 \qquad (w_{ip} = 0). \tag{1}$$

3.2 一人一枠制約

各参加者への割当は高々一つである。

$$\sum_{i=1}^{m} x_{ip} \le 1 \qquad (\forall p \in P). \tag{2}$$

「高々一つ」であって「必ず一つ」ではないので、希望が衝突して全員に割り当てられない場合にも、実行可能な部分割当を求められる。

3.3 共有資源の重複禁止

任意の時刻 t において、一つの時間枠しか使用できない。

$$\sum_{i:\, s_i \le t < e_i} \sum_{p \in P} x_{ip} \le 1 \qquad (\forall t). \tag{3}$$

この制約は、同じ行を 2 人に割り当てることも、異なる行の重複枠を同時に採用することも禁じる。実際の計算では無限個の時刻を列挙せず、区間の端点関係を使う。

同等の有限な制約として、同一時間枠に対して

$$\sum_{p \in P} x_{ip} \le 1 \qquad (\forall i)$$

を置き、異なる重複区間 $J_i \cap J_j \neq \varnothing$ に対して

$$\sum_{p \in P} x_{ip} + \sum_{p \in P} x_{jp} \leq 1$$

を置くこともできる。原コードは明示的な整数計画ソルバを用いず、これらの制約を DP の構造によって守る。

3.4 割当を評価する四次元ベクトル

割当 x に対し、以下を定義する。

$$N(x) = \sum_{i,p} x_{ip}, \qquad \text{割当人数,} \tag{4}$$

$$C_1(x) = \sum_{i,p} \mathbf{1}[w_{ip} = 1]\ x_{ip}, \qquad \text{第 1 希望の割当数,} \tag{5}$$

$$C_2(x) = \sum_{i,p} \mathbf{1}[w_{ip} = 2]\ x_{ip}, \qquad \text{第 2 希望の割当数.} \tag{6}$$

ここで $\mathbf{1}[Q]$ は命題 Q が真なら 1、偽なら 0 となる**指示関数**である。

時間枠の開始日時が早い順に、$0, 1, \ldots, m-1$ の順位 r_i を付与する。開始日時が一致したときは終了日時、元の行番号の順で順位が決まる。順位和を

$$R(x) = \sum_{i,p} r_i x_{ip}$$

とする。最終的な目的関数は

$$\boxed{\text{lexmax}_x\ \Phi(x), \qquad \Phi(x) = \big(N(x), C_1(x), C_2(x), -R(x)\big)}. \tag{7}$$

3.5 辞書式順序（lexicographic order）の意味

二つのベクトル $a, b \in \mathbb{Z}^4$ を左から比較し、最初に相違が現れる成分が大きい方を優位とする。例えば

$$(8, 2, 4, -20) >_{\text{lex}} (7, 7, 0, -3)$$

である。第 1 希望が少なくても、割当人数が多い解を優先する。また

$$(8, 5, 1, -40) >_{\text{lex}} (8, 4, 4, -2)$$

である。割当人数が同じなら、第 1 希望の数が優先される。

注意 3.1 (第 3 希望の役割). 第 3 希望の件数は明示的には数えない。すべての割当は第 1・第 2・第 3 希望のいずれかなので

$$C_3(x) = N(x) - C_1(x) - C_2(x).$$

したがって N と C_1 を固定して C_2 を最大化すると、C_3 は最小になる。これは第 3 希望を禁止することを意味しない。人数最大化に必要なら第 3 希望も採用する。

注意 3.2 (時刻評価は総順位和). $-R(x)$ は「最も早い開始時刻を優先する」という単純な辞書順ではない。採用した時間枠全体の**開始日時順位の合計**を最小化する。また、実時間の分数や日数の合計ではなく、希望の存在する枠の**相対順位**で評価する。同順位の希望充足度を持つ解の最後のタイブレークとしてのみ使われる。

3.6 重み付き和との違い

例えば $N + 0.1C_1 + 0.01C_2$ のように重みを付ける場合、最大値の上下限を吟味しなければ、下位基準が上位基準を逆転する危険がある。辞書式目的関数は優先順位を直接定義するので、その危険がない。逆に、このルールには「第 1 希望を何件増やせば未割当 1 人を容認できるか」というトレードオフが存在しない。この優先順位が運用方針に適合するかは、数学とは別の設計判断である。

4 区間スケジューリングと動的計画法

4.1 なぜ単純な貪欲法では不足するのか

開始時刻の早い枠から順に割り当てる方法では、後続の枠を圧迫することがある。希望順位が高いものから採用する方法でも、その時点で使わなかった枠の組合せによって全体の割当人数が変わるため、必ずしも大域的最適解を得られない。

時間区間の相互排他性は、ある枠を選ぶと利用できる前後の候補を制限する。さらに各人に高々 1 枠という制約が加わる。原実装は**区間の終了時刻による *DP* と参加者集合の状態管理**を組み合わせることで、この二つを同時に扱う。

4.2 終了時刻順へのソート

希望者が一人以上存在する m 個の区間を

$$e_1 \le e_2 \le \cdots \le e_m$$

となるように並べる。終了時刻が等しい場合は開始時刻、元の入力行番号で順序を決める。この番号は説明用の**内部番号**であり、結果表示のための元の行番号とは異なる。

4.3 直前の両立可能位置

各区間 j に対して

$$p(j) = \max\bigl(\{k \mid 1 \le k < j,\ e_k \le s_j\} \cup \{0\}\bigr) \tag{8}$$

と定義する。$p(j)$ は、現在の区間 j の開始以前に終了する、最も後ろの内部番号である。終了時刻順に整列済みなので、$1, \ldots, p(j)$ のすべての区間は j と両立する。

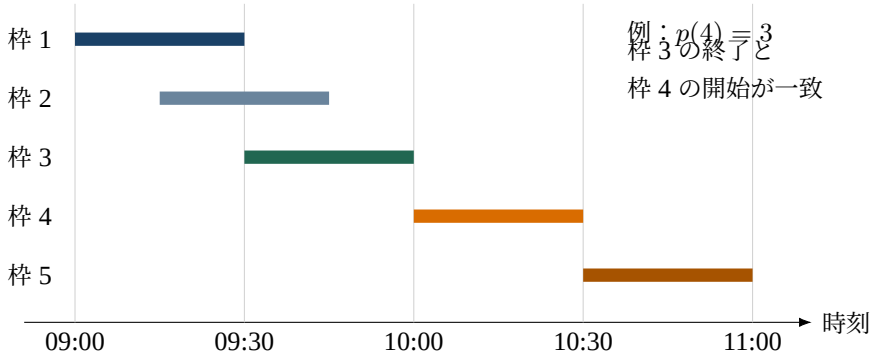


図 1: 終了時刻順の区間と $p(j)$：隣接区間は同時採用可能

4.4 参加者集合をビットマスクで表す

参加者 p にビット番号 p を割り当て、部分集合 $M \subseteq P$ を

$$\text{bits}(M) = \sum_{p \in M} 2^p \tag{9}$$

で表す。3 人 A,B,C の例を表 2 に示す。

表 2: ビットマスクと割当済み集合

整数値	二進数（CBA）	割当済み集合 M
0	000	$\varnothing$
1	001	$\{A\}$
2	010	$\{B\}$
3	011	$\{A, B\}$
4	100	$\{C\}$
5	101	$\{A, C\}$
6	110	$\{B, C\}$
7	111	$\{A, B, C\}$

Python では次のビット演算に対応する。

```
bit = 1 << person                # 2**person
already_assigned = bool(mask & bit)
next_mask = mask | bit           # add this person
```

`mask & (1 << person)` が非ゼロなら、その人は割当済みである。ビット集合により、集合の所属判定と追加を整数演算で実現できる。

4.5 DP 状態の厳密な意味

定義 4.1 (部分問題の評価値)**.** $F_j(M)$ を、**終了時刻順で最初の j 個の区間のみを使用して、割当済み参加者がちょうど M となる実行可能解**のうち、目的関数 Φ が辞書式に最大のものの評価値とする。実行不可能な場合は $F_j(M) = -\infty$ とする。

ここで「ちょうど M」が重要である。割当人数が同じでも、どの参加者が割当済みかによって今後使用できる参加者が異なる。この情報を残さず評価値だけ保存すると、誤って同じ人に二つの枠を割り当てる可能性がある。

初期条件は

$$F_0(\varnothing) = (0, 0, 0, 0), \qquad F_0(M) = -\infty \quad (M \neq \varnothing) \tag{10}$$

である。

4.6 区間一件を採用するときの増分

区間 j を参加者 q に割り当てることが許されるのは $w_{jq} > 0$ のときだけである。この割当による評価値の増分を

$$\Delta(j, q) = \left(1, \mathbf{1}[w_{jq} = 1], \mathbf{1}[w_{jq} = 2], -r_j\right) \tag{11}$$

とする。区間を採用すると人数成分は必ず 1 増え、希望順位に応じた成分が増え、最後の成分には開始日時の順位が減算される。

4.7 DP の基本漸化式

現在の区間 j に注目し、割当済み集合を M とする。実行可能解は「区間 j を使用しない」か「誰か $q \in M$ に割り当てる」かの二択である。前者は $F_{j-1}(M)$ に対応する。後者の場合、その他の区間はすべて j の開始以前に終了しなければならないので、残りの最適解は $F_{p(j)}(M \setminus \{q\})$ から構成できる。

したがって

$$\boxed{F_j(M) = \operatorname{lexmax} \left\{ \begin{array}{l} F_{j-1}(M), \\ F_{p(j)}(M \setminus \{q\}) + \Delta(j, q) \quad (q \in M,\ w_{jq} > 0) \end{array} \right\}} \tag{12}$$

となる。ただし、条件を満たさない候補と実行不可能な状態は除外する。

注意 4.2 (古典的区間スケジューリングとの関係)**.** 通常の重み付き区間スケジューリングでは、$\mathrm{OPT}(j) = \max\{\mathrm{OPT}(j-1), v_j + \mathrm{OPT}(p(j))\}$ の形となる。本実装では OPT が部分集合 M ごとに存在し、重み v_j が参加者と希望順位に依存する四次元ベクトルになっている。

4.8 正方向の更新として見る

式 (12) は、原コードと同じく**正方向**にも書ける。まず

$$F_j(M) \leftarrow F_{j-1}(M)$$

として区間 j を使わない解を継承する。次に各 M について $F_{p(j)}(M)$ を参照し、未割当の参加者 $q \notin M$ かつ $w_{jq} > 0$ なら

$$F_j(M \cup \{q\}) \leftarrow \max_{\text{lex}} \left\{ F_j(M \cup \{q\}), F_{p(j)}(M) + \Delta(j, q) \right\} \tag{13}$$

と更新する。この式が `solve()` のループに直接対応する。

5 Python 実装と数学記号の対応

5.1 前処理と入力検査

原実装は人数 $n > 15$ を拒否し、入力配列の長さと希望値の型・値を検査する。各行の希望値がすべてゼロなら、その行を最適化対象から除外する。

```
if n > 15:
    raise ValueError(...)
if not (len(dates) == len(slots) == len(wishes)):
    raise ValueError(...)
for i, (d, s, preferences) in enumerate(...):
    if not any(preferences):
        continue
    start, end = _bounds(d, s)
    events.append((start, end, i, preferences))
```

その後、`events.sort(key=lambda x: (x[1], x[0], x[2]))` によって終了時刻順に並べる。希望順位のタイブレークに使う r_j は別の開始時刻順（開始、終了、元行番号）で算出される。***DP* の処理順と、早さを評価する順位の定義は異なる**。

5.2 DP テーブルの初期化

原実装では `dp[j]` は辞書であり、そのキーはビットマスク整数、値は `(score, assignments)` である。

```
dp = [
    {0: ((0, 0, 0, 0), ())}
]
```

score は数理モデルの $F_j(M)$、assignments は最良値を実現する具体的な割当の復元情報である。割当は (original_slot_index, person_index) のタプルとして記録される。

5.3 $p(j)$ に対応する探索

```
available = 0
for k in range(j - 1, 0, -1):
    if events[k - 1][1] <= start:
        available = k
        break
```

Python のリストは 0 始まり、説明の区間番号は 1 始まりである。このため events[k-1] が数学上の区間 k に対応する。探索結果 available が $p(j)$ である。

5.4 状態遷移と辞書式比較

```
current = dp[-1].copy()
for mask, (score, assignments) in dp[available].items():
    for person, rank in enumerate(prefs):
        if rank == 0 or (mask & (1 << person)):
            continue
        next_mask = mask | (1 << person)
        candidate = (
            score[0] + 1,
            score[1] + (rank == 1),
            score[2] + (rank == 2),
            score[3] - order[idx]
        )
        existing = current.get(next_mask)
        if existing is None or candidate > existing[0]:
            current[next_mask] = (
                candidate, assignments + ((idx, person),)
            )
dp.append(current)
```

rank == 1 は Python で真なら True（算術上は 1）、偽なら False（算術上は 0）となる。4 要素タプルの > は左から成分を比較するので、辞書式目的関数 Φ の順序をそのまま実現する。

なお current = dp[-1].copy() は区間を使わない候補を継承する操作である。dp[available] は、現在の区間と時間が重ならない過去の最適解である。二種類の遷移が同時に実装されている。

5.5 最終解と復元

```
best_mask, (best_score, choices) = max(
    dp[-1].items(),
```

```
    key=lambda item: (item[1][0], -item[0])
)
```

最終段階で到達可能なビットマスクすべてを比較し、評価値が最大の状態を選ぶ。評価値が同一なら $-\texttt{mask}$ により**ビットマスク整数が小さい方**を選ぶ。この最後の規則は希望順位とは別の、実装上の決定的なタイブレークである。同じビットマスク・同じ評価値に対して複数経路が存在するときは、先に記録された割当が残る（更新条件が `>` であり `>=` ではないため）。

最終結果は `Solution(assignments,unassigned,counts)` であり、第 3 希望の件数は $N - C_1 - C_2$ により計算する。

6　実際に手で追う計算例

6.1　入力例と両立位置

3 名 A、B、C と 5 時間枠を考える。全枠の日付は同一とする。表 3 の r_j は開始時刻順の 0 始まり順位である。

表 3: 具体例：希望行列、開始順位、両立可能位置

j	時間枠	A	B	C	r_j	$p(j)$
1	09:00–09:30	1	0	0	0	0
2	09:15–09:45	0	1	0	1	0
3	09:30–10:00	0	2	1	2	1
4	10:00–10:30	1	1	2	3	3
5	10:30–11:00	0	0	1	4	4

枠 1 と枠 2 は重なっている。一方、枠 1 と枠 3、枠 3 と枠 4、枠 4 と枠 5 は、境界で接するだけなので両立する。

6.2　第 1 段階：枠 1 まで

開始状態は $F_0(\varnothing) = (0, 0, 0, 0)$ である。枠 1 は A のみが第 1 希望であるため、

$$F_1(\varnothing) = (0, 0, 0, 0), \qquad F_1(\{A\}) = (1, 1, 0, 0)$$

が得られる。後者のビットマスクは二進数 `001`、整数値 1 である。

6.3 第 2 段階：枠 2 まで

枠 2 は B が第 1 希望だが、枠 1 と重なっている。$p(2) = 0$ なので、枠 2 を採用する解は F_0 から構成しなければならない。したがって

$$F_2(\{B\}) = (1, 1, 0, -1)$$

は作れるが、枠 1 の A と枠 2 の B を同時に採用する状態 $\{A, B\}$ は作れない。この段階で「それぞれ別人なのだからよい」という誤った割当を防いでいる。

6.4 第 3 段階：枠 3 まで

$p(3) = 1$ であるため、枠 3 は枠 1 と両立する。枠 3 を C の第 1 希望として割り当てると

$$\begin{aligned} F_3(\{A, C\}) &= F_1(\{A\}) + \Delta(3, C) \\ &= (1, 1, 0, 0) + (1, 1, 0, -2) \\ &= (2, 2, 0, -2) \end{aligned}$$

となる。一方、枠 3 を B の第 2 希望として割り当てると

$$F_3(\{A, B\}) = (1, 1, 0, 0) + (1, 0, 1, -2) = (2, 1, 1, -2)$$

となる。この時点では、A と C を割り当てる方が第 1 希望件数で優れる。

6.5 第 4 段階：枠 4 まで

$p(4) = 3$ である。枠 3 までの状態 $\{A, C\}$ に B の第 1 希望である枠 4 を追加できる。

$$\begin{aligned} F_4(\{A, B, C\}) &\succeq_{\text{lex}} F_3(\{A, C\}) + \Delta(4, B) \\ &= (2, 2, 0, -2) + (1, 1, 0, -3) \\ &= (3, 3, 0, -5). \end{aligned}$$

つまり

$$\boxed{\text{A} \mapsto 1, \quad \text{C} \mapsto 3, \quad \text{B} \mapsto 4}$$

という割当で、全員を第 1 希望に割り当てられる。

別の候補として、枠 2 を B、枠 4 を A、枠 5 を C に与えても全員が第 1 希望になる。しかし順位和は $1 + 3 + 4 = 8$ であり、選ばれた割当の $0 + 2 + 3 = 5$ より大きい。したがって辞書式評価では

$$(3, 3, 0, -5) >_{\text{lex}} (3, 3, 0, -8)$$

となる。希望充足数に差がないため、初めて時間の早さが比較される。

6.6 最終段階：枠 5 まで

枠 5 を採用する候補は存在するが、すでに得られた $(3, 3, 0, -5)$ を上回らない。よって最終解は

$$\Phi^* = (3, 3, 0, -5), \qquad \text{未割当者} = \varnothing$$

である。コード上の戻り値は、入力行番号が 0 始まりなので

```
Solution(
    assignments=((0, 0), (2, 2), (3, 1)),
    unassigned=(),
    counts=(3, 3, 0)
)
```

と表される。

6.7 重要な教訓：局所的に最善とは限らない

枠 2 は B の第 1 希望であるが、これを先に確定してしまうと枠 1 の A を失う。しかし枠 3 の C と枠 4 の B を組み合わせれば、全員が第 1 希望になる。このような全体の組合せは、局所的な希望順位だけを見るアルゴリズムでは見つけられない。

7 厳密最適性の証明

7.1 準備：延長可能性を決める情報

補題 7.1 (同一状態の支配関係)**.** 同じ j と同じ割当済み集合 M に対応する二つの実行可能解 X, Y を考える。$\Phi(X) \geq_{\text{lex}} \Phi(Y)$ なら、*DP* で Y を破棄し X のみ保存しても、将来の最適解を失わない。

Proof. 未来の区間に新たに割り当てられる参加者は $P \setminus M$ のみである。同じ M に対して、将来使用可能な参加者集合は一致する。また、ある未来の区間 k へ $F_j(M)$ を延長する遷移が許されるのは $p(k) \geq j$ の場合であり、このとき $1, \ldots, j$ のどの区間も k と時間的に両立する。$p(k) < j$ の場合、遷移はそもそも $F_j(M)$ を参照しない。従って、同じ段階と集合を持つ二つの状態は、その先の許容される延長先についても一致する。同じ追加割当を X, Y に施した場合、その評価増分は同一のベクトル d である。辞書式順序はベクトル加算と両立するので $\Phi(X) + d \geq_{\text{lex}} \Phi(Y) + d$ が成り立つ。したがって劣る Y を保存する必要はない。 □

定理 7.2 (DP 漸化式の正当性)**.** 式 (10) を初期条件とし、式 (12) により計算した $F_j(M)$ は、最初の j 区間からちょうど参加者集合 M を割り当てるすべての実行可能解の中で、辞書式目的関数を最大にする。

Proof. j に関する数学的帰納法を用いる。$j = 0$ では、区間を一つも選べないため $M = \varnothing$ の解だけが実行可能であり、初期条件は正しい。

$0, 1, \ldots, j-1$ に対して主張が成立すると仮定する。任意の実行可能解 X が区間 j を採用しない場合、X は最初の $j-1$ 区間から構成され、帰納法の仮定よりその最良値は $F_{j-1}(M)$ である。

区間 j を参加者 q に割り当てる場合、他の採用区間 i はすべて $e_i \le s_j$ を満たす。そうでなければ、終了時刻順に $i < j$ であることと合わせて i と j が重複してしまう。従って他の採用区間はすべて $1, \ldots, p(j)$ に含まれ、その割当済み集合は $M \setminus \{q\}$ である。帰納法の仮定より、この部分の最良値は $F_{p(j)}(M \setminus \{q\})$ である。さらに $w_{jq} > 0$ という希望条件と $q \notin M \setminus \{q\}$ を満たし、この部分解に区間 j を追加すれば実行可能な解を構成できる。よって、区間 j を採用する場合の最良値は式 (12) の第 2 候補で尽くされる。

どの実行可能解もこの二場合のいずれかであり、両場合から得られる候補も実行可能である。これらの辞書式最大値を取る漸化式は正しい。 □

定理 7.3 (ソルバの大域的最適性)**.** 前処理で得られた区間集合と希望行列が有効であるとき、`solve()` が返す割当は、式 (1)、(2)、(3) を満たし、式 (7) の目的関数を大域的に最大化する。

Proof. 前定理より、各 M の $F_m(M)$ は、その集合を割り当てる最良値である。すべての実行可能解には何らかの割当済み集合 M が対応する。よって $\mathrm{lexmax}_{M \subseteq P} F_m(M)$ は実行可能解全体の最大値である。最終処理の `max(dp[-1].items(), ...)` はまさにこれを求め、保存した復元情報から実行可能な割当を返す。 □

注意 7.4 (最適性の適用範囲)**.** これは**定式化された制約と評価基準に対する最適性**である。移動・準備・休憩時間、参加者の属性間の公平性、希望枠外での調整可能性、運用上の優先者など、モデルに与えていない条件についての最適性は意味しない。

8 計算量とスケーラビリティ

8.1 状態数

参加者数を n とすると、割当済み集合 M の候補は最大 2^n 通りである。時間枠の段階は $m+1$ 通りなので、DP 辞書が保持し得る状態数の粗い上界は

$$(m+1)2^n$$

である。実際には、区間の個数より多い人を割り当てられないこと、希望が存在しない組合せがあること等により、到達可能な状態は少なくなることもある。

8.2 時間計算量

前処理のソートは $O(m \log m)$ である。$p(j)$ を各区間で後ろ向きに線形探索するので、その合計は最悪 $O(m^2)$ となる。DP では、各段階に最大 2^n 個の状態があり、各状態から最大 n 人を試すため、評価ベクトルと復元情報を定数時間で更新できると仮定した**抽象的な遷移数**は

$$O(mn2^n)$$

である。したがって、基本的な演算数の評価は

$$O(m \log m + m^2 + mn2^n) = O(m^2 + mn2^n). \tag{14}$$

ただし原コードの `assignments + ((idx, person),)` は既存タプルをコピーして新タプルを生成する。割当履歴の長さは最大 n なので、各遷移が最悪 $O(n)$ かかる。Python 実装の履歴コピーも含めた安全側の上界は

$$\boxed{O(m^2 + mn^2 2^n)} \tag{15}$$

と評価できる（ソート項は省略）。同様に、辞書本体の状態数だけなら $O(m2^n)$ だが、履歴タプルの実体まで含めれば空間計算量の粗い上界は $O(mn2^n)$ である。

8.3 人数 15 人という制限の意味

n	2^n
5	32
10	1,024
15	32,768
20	1,048,576

参加者数を 5 増やすごとに状態集合の最大数が 32 倍になる。したがって原実装は $n > 15$ を拒否する。これは *15* **人なら必ず高速**という保証ではなく、指数的増大を抑えるための実務上の上限である。

8.4 同じ厳密性を保った改善案

1. 終了時刻列に対して `bisect` を用い、$p(j)$ を各 $O(\log m)$ で算出する。
2. DP 状態には評価値と直前状態へのポインタだけを保持し、復元用の長いタプルを毎回コピーしない。
3. 前処理で明らかに希望を持たない枠を除外する現行方針を保ち、到達可能なビットマスクのみ辞書に保存する。
4. 必要なら、同等の目的関数を持つ整数計画モデルで小規模入力の結果を照合する。

いずれも元の目的関数を変更せずに改善できる可能性があるが、実測性能は問題例の希望密度と区間の重なり方に依存する。

9 画面処理・ファイル処理の読み方

9.1 Qt のイベント駆動

`Scheduler(QMainWindow)` は表、ボタン、メニュー、結果欄を配置する。例えば

```
self.table.cellDoubleClicked.connect(self.toggle_wish)
button.clicked.connect(callback)
```

によって、Qt の**シグナル**と Python の**スロット（コールバック）**を結び付ける。ダブルクリックされた行・列が `toggle_wish(row,col)` に渡され、希望値が $(w+1) \bmod 4$ で循環する。

9.2　枠の一括生成

`BatchSlotDialog` は開始・終了を分単位の整数に変換し、長さ h 分の半開区間を連続して生成する。$[a, b)$ をちょうど h 分ずつ区切るには

$$b > a, \qquad (b-a) \bmod h = 0$$

が必要である。この条件を満たさない場合、端数枠を作らず入力を拒否する。

9.3　希望の可視化と実際の割当は別物

`refresh()` は各時間枠の希望人数を数え、希望なしを灰、希望 1 人を緑、希望 2 人以上を橙で表示する。ソルバによって実際に採用された**参加者セル**は青色となる。

緑色は「その行を希望する人が一人」という局所的事実であって、ソルバによる確定割当を意味しない。異なる行の時間重複は、この局所的な色分けに反映されないが、`solve()` の時間制約では考慮される。

9.4　JSON 保存と復元

`payload()` は `version`, `people`, `dates`, `slots`, `wishes` を JSON 化する。日本語をそのまま出力するため `ensure_ascii=False` を指定する。保存処理は隣接する一時ファイル `.tmp` に書き込み、それを目的パスへ置換する。これは直接書き換える場合と比べて、途中失敗による部分書込の危険を減らす。

原コードは**ソルバ結果を保存しない**。ファイルを再度開いた場合には、入力希望から再計算する。古い形式の真偽値希望は `True -> 1`, `False -> 0` として読み替える。日付のない旧形式の行は空日付として取り扱われ、計算時には西暦 1 年 1 月 1 日へ写像される。

10　入力検証・限界・モデル拡張

10.1　現在の実装が明示的に保証すること

1. 希望のない組合せを採用しない。
2. 一人につき高々 1 枠を採用する。
3. 共有資源上の時間的重複を認めない。

4. 制約内で割当人数、第 1 希望数、第 2 希望数、早い時間順位和の順に最適化する。
5. 全員割当が不可能な場合にも、人数最大の部分割当を返す。

10.2 現在の実装が保証しないこと

1. 各参加者の第 1・第 2・第 3 希望が一つずつであること。複数入力は許容される。
2. 面談間の移動時間、準備時間、休憩時間などの自動確保。
3. 複数の会場や複数教員を同時に使用する資源割当。
4. 希望入力のない参加者を無理に割り当てること。
5. 異なる日付・時刻をまたぐ夜間区間の入力。
6. 登録した枠のすべてが非重複であること。異なる枠の重複は許され、ソルバ側で同時採用を防ぐ。

10.3 入力検査に関する実装上の注意

`solve()` は希望のある各行について日付・時刻文字列を解析する。しかし、**希望が全くない行**は解析前にスキップするので、その行の日時文字列が不正でもソルバではエラーにならない場合がある。また `open_file()` は JSON の構造・日付を検査するが、時間枠文字列の完全な解析と区間の正当性確認は読込時ではなく主にソルバ実行時に委ねられる。

この違いを意識し、厳格なデータ検証が必要ならファイル読込時と編集確定時にも`_bounds()` 相当のチェックを統一して適用することが望ましい。

10.4 公平性・必須対象者・時間的余裕の導入（拡張案）

以下は**原実装には存在しない**モデル拡張である。

■必須参加者　必須対象者の集合 $P_{\text{req}} \subseteq P$ を設け、

$$\sum_{i} x_{ip} = 1 \qquad (p \in P_{\text{req}})$$

とすると、割当不可能な入力に対しては実行可能解自体が存在しなくなる。この場合には、制約充足失敗を明示的に報告する設計が必要になる。

■準備時間　面談終了後に δ 分の準備時間が必要なら、各区間を

$$J_i' = [s_i, e_i + \delta)$$

のように拡張して衝突判定する方法がある。ただし日付をまたぐか否か、次の区間の始まりにのみ余裕が必要かなど、運用規則を確認しなければならない。

■希望順位の個人差　第 1 希望の件数だけを最大化すると、同一人物が常に不利な希望に回ることの不公平性は評価できない。公平性を目的にする場合は、個人ごとの選好効用や所属グループの充足率を追加する必要がある。その際は元の辞書式優先関係との整合性を再検討しなければならない。

■複数資源　面談可能な教員や部屋が複数ある場合は、割当変数を x_{ipr}（時間枠 i、参加者 p、資源 r）へ拡張できる。しかし単一資源という区間 DP の構造が変わるため、本コードをそのまま使えるとは限らない。

11　検証方法：厳密解の独立確認

11.1　小規模な全列挙との照合

DP の正当性は上記の帰納法で説明できるが、実装ミスの検出には別方式の検証が有効である。小規模な n, m について、希望のある候補 (i, p) から部分集合を全列挙し、一人一枠・時間非重複の条件を満たすものだけを残して Φ を計算すれば、独立した比較基準を作れる。

この資料の作成に際しては、添付ファイルから取り出した `solve()` と、独立に記述した全列挙評価を用いて、参加者 1–4 人・時間枠 0–7 件の乱数生成問題 500 例で評価ベクトルの一致を確認した。これは実装の補助的な検証であり、**すべての入力に対する形式的証明の代替ではない**。

11.2　最低限確認したい境界事例

1. 参加者ゼロ、または時間枠ゼロ（API から直接呼ぶ場合）。
2. すべての希望が 0 である場合。
3. ちょうど接する区間 $[a, b)$ と $[b, c)$。
4. 部分的に重なる区間 $[a, c)$ と $[b, d)$（$a < b < c < d$）。
5. 希望が同じ参加者に集中した場合。
6. 割当人数は同数で第 1 希望の件数だけが異なる場合。
7. 上位 3 成分が一致し、開始順位和だけが異なる場合。
8. 元の配列順と終了時刻順が一致しない場合。
9. 複数日にまたがる時間枠と、空日付を持つ旧形式データ。

12　より一般的な視点とまとめ

本問題は、**区間の両立性**という時間軸上の組合せ構造に、**参加者は一度しか選べない**という集合的制約を組み合わせたものである。数理的には、希望のある参加者・時間枠の組を候補とし、時間競合と参加者重複を禁止した集合選択問題と見ることもできる。

この構造に対し、終了時刻順の前処理で「時間的にどこまで戻れるか」を $p(j)$ に集約し、ビット

マスク M に「誰が既に使われたか」を集約している。その結果、過去の割当全履歴を DP の**状態の判定**に用いる必要がなくなり、評価値の優越関係により多数の劣る候補を捨てられる。

$$\underbrace{\text{終了時刻順の区間 DP}}_{\text{時間的両立性}} \quad + \quad \underbrace{\text{ビットマスク}}_{\text{参加者の一意性}} \quad + \quad \underbrace{\text{辞書式評価}}_{\text{運用上の優先順位}}$$

最も重要な設計上の論点は、アルゴリズムの選択以前に「何を優先し、何を絶対に許さないか」を明確に定式化することである。原実装の厳密性は、その定式化に対するものである。変更したい運用方針があるなら、まず制約式または目的関数の変更として記述し、その後に適したアルゴリズムを検討するのが望ましい。

付録 A　記号一覧

記号	意味
P, n	参加者集合、参加者数
I, ℓ	元の時間枠集合、時間枠数
m	希望のある時間枠数
s_i, e_i	時間枠 i の開始・終了日時
J_i	時間枠 i の半開区間 $[s_i, e_i)$
w_{ip}	時間枠 i に対する参加者 p の希望順位
x_{ip}	時間枠 i を参加者 p に割り当てる二値変数
r_i	開始日時順に付けた 0 始まり順位
N, C_1, C_2	割当人数、第 1 希望件数、第 2 希望件数
$R(x)$	採用した枠の開始順位和
$\Phi(x)$	辞書式評価ベクトル $(N, C_1, C_2, -R)$
$p(j)$	区間 j の開始以前に終了する直前の区間番号
M	割当済み参加者の集合
bits(M)	集合 M のビットマスク整数
$F_j(M)$	最初の j 区間でちょうど M を割り当てる最良値
$\Delta(j, q)$	区間 j を参加者 q に割り当てる評価増分

付録 B　読解する際のソースコード位置

原ファイルの行範囲	内容
34–46	`Solution` と結果型
49–65	日時解析 `_bounds()`
68–146	厳密最適化 `solve()`
159–209	単一時間枠の編集ダイアログ
212–283	一括時間枠生成ダイアログ
286–436	メイン画面の初期化・入力処理・一覧表示
438–562	参加者・時間枠操作とソルバ結果表示
564–670	JSON 保存・復元と終了処理
673–682	アプリケーション起動